```
pcm1: <Conexant CX20561 (Hermosa) (Internal Analog Mic)> at nid 29 on hdaa0
```

The status of the sound card may also be checked using this command:

```
# cat /dev/sndstat
```

The output should be similar to the following:

```
Installed devices:
pcm0: <Conexant CX20561 (Hermosa) (Analog 2.0+HP/2.0)> (play/rec) default
pcm1: <Conexant CX20561 (Hermosa) (Internal Analog Mic)> (rec)
```

If no `pcm` devices are listed, double-check that the correct device driver was loaded. If all goes well, the sound card should now work in FreeBSD.

beep(1) can be used to produce some noise, confirming that the sound card is working:

```
% beep
```

9.2.2. Mixer

FreeBSD has different utilities to set and display sound card mixer values built on the FreeBSD Sound System:

Table 17. Supported mixer packages

Name	License	Package	Toolkit
mixer(8)	BSD-2	Included in base system	CLI
dsbmixer	BSD-2	audio/dsbmixer	Qt
KDE Plasma audio widget	GPL 2.0	audio/plasma6-plasma-pa	Qt
mixertui	BSD-2	audio/mixertui	TUI

9.2.3. Graphics Card Sound

Graphics cards often come with their own integrated sound devices, and it may be unclear which is being used as the default device. To confirm, run dmesg and look for the pcm entries to identify how the system is enumerating the outputs. Execute the following command:

```
% dmesg | grep pcm
```

The output looks something like this:

```
pcm0: <HDA NVIDIA (Unknown) PCM #0 DisplayPort> at cad 0 nid 1 on hdac0
pcm1: <HDA NVIDIA (Unknown) PCM #0 DisplayPort> at cad 1 nid 1 on hdac0
pcm2: <HDA NVIDIA (Unknown) PCM #0 DisplayPort> at cad 2 nid 1 on hdac0
pcm3: <HDA NVIDIA (Unknown) PCM #0 DisplayPort> at cad 3 nid 1 on hdac0
hdac1: HDA Codec #2: Realtek ALC889
pcm4: <HDA Realtek ALC889 PCM #0 Analog> at cad 2 nid 1 on hdac1
pcm5: <HDA Realtek ALC889 PCM #1 Analog> at cad 2 nid 1 on hdac1
pcm6: <HDA Realtek ALC889 PCM #2 Digital> at cad 2 nid 1 on hdac1
pcm7: <HDA Realtek ALC889 PCM #3 Digital> at cad 2 nid 1 on hdac1
```

The graphics card (NVIDIA®) has been enumerated before the sound card (Realtek®), with the sound card appearing as pcm4. The system can be configured to use the sound card as the default device by executing the following command:

```
# sysctl hw.snd.default_unit=4
```

To make this change permanent add the next line to /etc/sysctl.conf:

```
hw.snd.default_unit=4
```

9.2.4. Automatically Switching to Headphones

Some systems may struggle with switching between audio outputs, but fortunately FreeBSD allows automatic switchover to be configured in device.hints.

Identify how the system is enumerating the audio outputs by executing the following command:

```
% dmesg | grep pcm
```

The output looks something like this:

```
pcm0: <Realtek ALC892 Analog> at nid 23 and 26 on hdaa0
pcm1: <Realtek ALC892 Right Analog Headphones> at nid 22 on hdaa0
```

Add the following lines to /boot/device.hints:

```
hint.hdac.0.cad0.nid22.config="as=1 seq=15 device=Headphones"
hint.hdac.0.cad0.nid26.config="as=2 seq=0 device=speakers"
```



Keep in mind that these values are for the example indicated above. They may vary depending on the system.

9.2.5. Troubleshooting Sound

Some common error messages and their solutions:

Table 18. Common Error Messages

Error	Solution
`xxx: can't open /dev/dsp!`	Type `fstat | grep dsp` to check if another application is holding the device open. Noteworthy troublemakers are esound and KDE's sound support.

Programs using audio/pulseaudio might need to restart the audio/pulseaudio daemon for the changes in `hw.snd.default_unit` to take effect. Alternatively, audio/pulseaudio settings can be changed on the fly. pacmd(1) opens a command line connection to the audio/pulseaudio daemon:

```
# pacmd
Welcome to PulseAudio 14.2! Use "help" for usage information.
>>>
```

The following command changes the default sink to card number 4 as in the previous example:

```
set-default-sink 4
```



Do not use the `exit` command to exit the command line interface. That will kill the audio/pulseaudio daemon. Use `Ctrl` + `D` instead.

9.3. Audio players

This section introduces some of the software available from the FreeBSD Ports Collection which can be used for audio playback.

Table 19. Audio players packages

Name	License	Package	Toolkit
Elisa	LGPL 3.0	audio/elisa	Qt
GNOME Music	GPL 2.0	audio/gnome-music	GTK+
Audacious	BSD-2	multimedia/audacious	Qt
MOC (music on console)	GPL 2.0	audio/moc	TUI

9.3.1. Elisa

Elisa is a music player developed by the KDE community that strives to be simple and nice to use.

To install Elisa, execute:

```
# pkg install elisa
```

9.3.2. GNOME Music

GNOME Music is the new GNOME music playing application. It aims to combine an elegant and immersive browsing experience with simple and straightforward controls.

To install GNOME Music, execute:

```
# pkg install gnome-music
```

9.3.3. Audacious

Audacious is an open source audio player. As a descendant of XMMS, it provides music playback while maintaining efficient use of system resources.

To install Audacious, execute:

```
# pkg install audacious-qt6 audacious-plugins-qt6
```



Audacious supports OSS natively, but must be configured in the settings on the Audio tab.

9.3.4. MOC (music on console)

MOC (music on console) is a console audio player designed to be powerful and easy to use.

MOC plays smoothly, regardless of system or I/O load, because it handles the output buffer in a separate thread. It does not cause gaps between files, because the next file to be played is pre-cached while playing the current file.

To install MOC (music on console), execute:

```
# pkg install moc
```

9.4. Video players

This section introduces some of the software available from the FreeBSD Ports Collection which can be used for video playback.

Table 20. Video players packages

Name	License	Package	Toolkit
MPlayer	GPL 2.0	multimedia/mplayer	CLI
SMPlayer	GPL 2.0	multimedia/smplayer	Qt
VLC media player	GPL 2.0	multimedia/vlc	Qt
Kodi (XBMC)	GPL 2.0	multimedia/kodi	X11

9.4.1. MPlayer

MPlayer is a multimedia player and encoder suite which runs on many platforms and works on the command line. It plays a terrific number of different file formats and codecs including popular DivX, XviD, H.264 streams as well as DVD and SVCDs along with many popular audio codecs.

To install MPlayer, execute:

```
# pkg install mplayer
```

For examples of how MPlayer works see mplayer(1).

9.4.2. SMPlayer

SMPlayer intends to be a complete front-end for MPlayer, from basic features like playing videos, DVDs, and VCDs to more advanced features like support for MPlayer filters and more.

To install SMPlayer, execute:

```
# pkg install smplayer
```

9.4.3. VLC media player

VLC media player is a highly portable multimedia player for various audio and video formats (MPEG-1, MPEG-2, MPEG-4, DivX, mp3, ogg, and more) as well as DVD's, VCD's, and various streaming protocols. It can also be used as a server to stream in unicast or multicast in IPv4 or IPv6 on a high-bandwidth network. VLC also has the ability to transcode media on-the-fly for streaming or saving to disk.

To install VLC, execute:

```
# pkg install vlc
```

9.4.4. Kodi (XBMC)

Kodi (formerly known as XBMC) is a free and open source cross-platform media-player and entertainment hub. It allows users to play and view most videos, music, podcasts, and other digital media files from local and network storage media and the internet.

To install Kodi, execute:

```
# pkg install kodi
```

9.5. Conferencing and Meetings

A FreeBSD desktop environment can be used to join video conferences. This section will explain how to configure the webcam and which videoconferencing applications are supported on FreeBSD.

9.5.1. Setting Up the Webcam

To allow FreeBSD access to the webcam and perform its configuration it is necessary to install certain utilities:

- multimedia/webcamd is a daemon that enables the use of hundreds of different USB based webcam and DVB USB devices.
- multimedia/pwcview is an application that can be used to view the video stream of the webcam.

To install the required utilities, execute:

```
# pkg install webcamd pwcview
```

Enable the webcamd(8) service in `/etc/rc.conf` to start it at system boot:

```
# sysrc webcamd_enable=YES
```

The user must belong to the `webcamd` group. To add the user to `webcamd` group execute the following command:

```
# pw groupmod webcamd -m username
```

Since multimedia/webcamd needs the cuse(3) module this module must be loaded by executing the following command:

```
# kldload cuse
```

To load cuse(3) at system boot, execute the command:

```
# sysrc kld_list+=cuse
```

Once the utilities have been installed the list of available webcams can be shown with webcamd(8):

```
# webcamd -l
```

The output should be similar to the following:

```
webcamd [-d ugen0.2] -N SunplusIT-Inc-HP-TrueVision-HD-Camera -S unknown -M 0 ①
webcamd [-d ugen1.3] -N Realtek-802-11n-WLAN-Adapter -S 00e04c000001 -M 0
```

① Available webcam

Configure the available webcam executing the following command:

```
# sysrc webcamd_0_flags="-N SunplusIT-Inc-HP-TrueVision-HD-Camera" ①
```



Note here that if this is a plug-and-play USB webcam, changing the USB port to which it is connected will change the output from `webcamd -l`, specifically the device identifier, and the entry in rc.conf might need to be updated. To avoid this issue either use the device name (`-N` option) and/or match with the device serial number if known (`-S` option) as displayed in the output of `webcamd -l`. For static devices, for example integrated laptop cameras, you can use the device identifier (`-d` option).

The webcamd(8) service must be started by executing the following command:

```
# service webcamd start
```

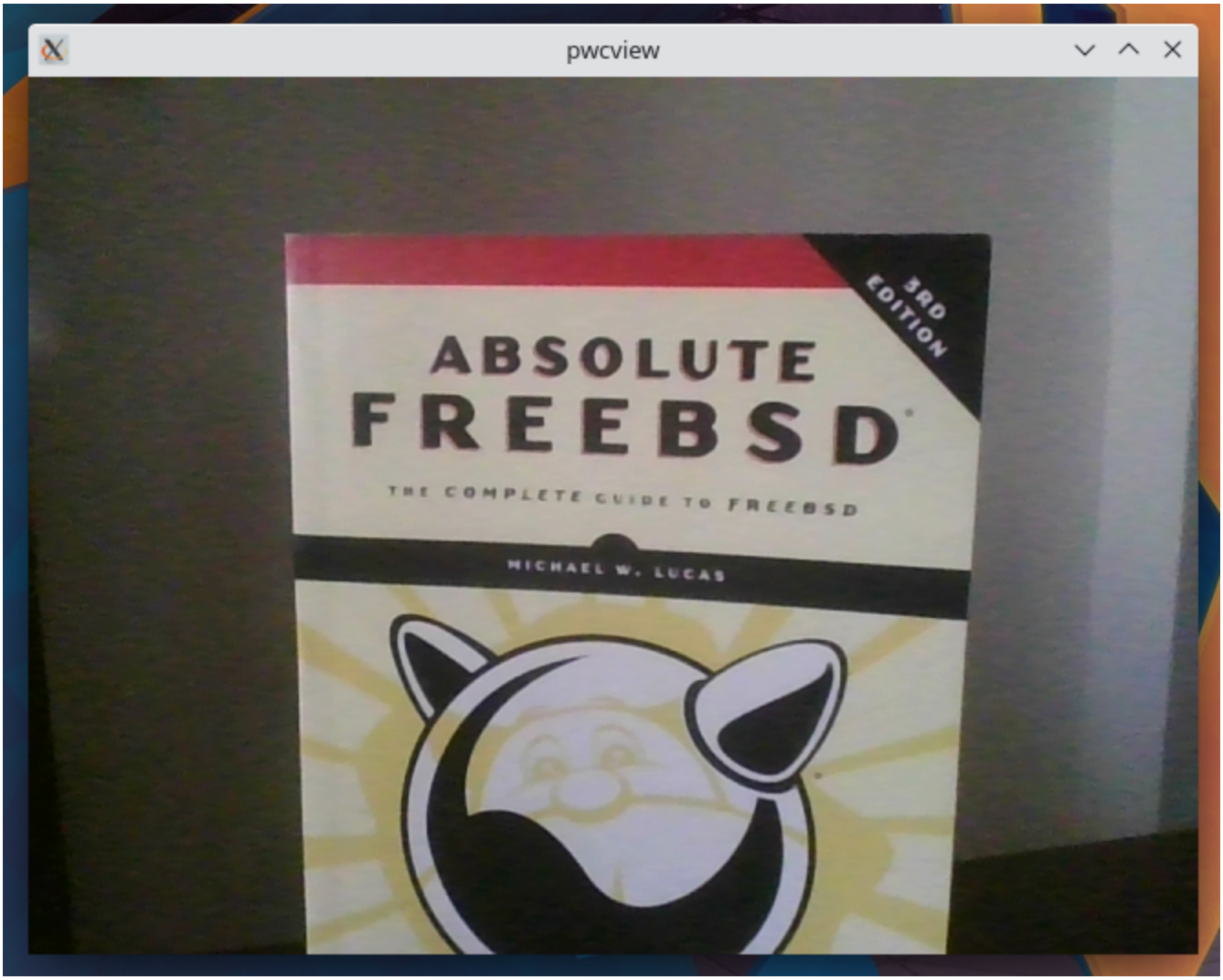
The output should be similar to the following:

```
Starting webcamd.
webcamd 1616 - - Attached to ugen0.2[0]
```

multimedia/pwcview can be used to check the proper functioning of the webcam. The following command can be used to execute multimedia/pwcview:

```
% pwcview -f 30 -s vga
```

Then multimedia/pwcview will display the webcam:



9.5.2. Meetings software status

FreeBSD currently supports the following tools used to carry out videoconferences.

Table 21. Meeting software

Name	Firefox status	Chromium status	Website
Microsoft Teams	Does not work	Works	https://teams.live.com
Google Meet	Works	Works	https://meet.google.com/
Zoom	Works	Works	https://zoom.us
Jitsi	Does not work	Works	https://meet.jit.si/
BigBlueButton	Does not work	Works	https://bigbluebutton.org/

9.6. Image Scanners

In FreeBSD, access to image scanners is provided by SANE (Scanner Access Now Easy), which is available in the FreeBSD Ports Collection.

9.6.1. Checking the Scanner

Before attempting any configuration it is important to check the scanner is supported by SANE.

With the scanner connected, run the following command to get all connected USB devices:

```
# usbconfig list
```

The output should be similar to the following:

```
ugen4.2: <LITE-ON Technology USB NetVista Full Width Keyboard.> at usbus4, cfg=0
md=HOST spd=LOW (1.5Mbps) pwr=ON (70mA)
ugen4.3: <Logitech USB Optical Mouse> at usbus4, cfg=0 md=HOST spd=LOW (1.5Mbps)
pwr=ON (100mA)
ugen3.2: <HP Deskjet 1050 J410 series> at usbus3, cfg=0 md=HOST spd=HIGH (480Mbps)
pwr=ON (2mA)
```

Run the following command to obtain the `idVendor` and the `idProduct`:

```
# usbconfig -d 3.2 dump_device_desc
```



Note here that the scanner is a plug-and-play device, and changing the USB port to which it is connected will change the output from `usbconfig list`.

The output should be similar to the following:

```
ugen3.2: <HP Deskjet 1050 J410 series> at usbus3, cfg=0 md=HOST spd=HIGH (480Mbps)
pwr=ON (2mA)

bLength = 0x0012
bDescriptorType = 0x0001
bcdUSB = 0x0200
bDeviceClass = 0x0000  <Probed by interface class>
bDeviceSubClass = 0x0000
bDeviceProtocol = 0x0000
bMaxPacketSize0 = 0x0040
idVendor = 0x03f0
idProduct = 0x8911
bcdDevice = 0x0100
iManufacturer = 0x0001  <HP>
iProduct = 0x0002  <Deskjet 1050 J410 series>
bNumConfigurations = 0x0001
```

Once the `idVendor` and the `idProduct` have been obtained, it is necessary to check in the list of supported devices of SANE if the scanner is supported by filtering by the idProduct.

9.6.2. SANE Configuration

SANE provides the access to the scanner via backends. To be able to scan with FreeBSD the graphics/sane-backends package must be installed by running the following command:

```
# pkg install sane-backends
```



Some USB scanners require firmware to be loaded. Like the HP scanner used in the example above, which needs the package print/hplip installed.

After installing the necessary packages devd(8) must be configured to allow FreeBSD access to the scanner.

Add the `saned.conf` file to /usr/local/etc/devd/saned.conf with the following content:

```
notify 100 {
        match "system" "USB";
        match "subsystem" "INTERFACE";
        match "type" "ATTACH";
        match "cdev" "ugen[0-9].[0-9]";
        match "vendor" "0x03f0"; ①
        match "product" "0x8911"; ②
        action "chown -L cups:saned /dev/\$cdev && chmod -L 660 /dev/\$cdev";
};
```

① `vendor`: Is the idVendor obtained previously by running the `usbconfig -d 3.2 dump_device_desc` command.

② `product`: Is the idProduct obtained previously by running the `usbconfig -d 3.2 dump_device_desc` command.

After that devd(8) must be restarted by running the following command:

```
# service devd restart
```

The SANE backends include scanimage(1) which can be used to list the devices and perform an image acquisition.

Execute scanimage(1) with `-L` argument to list the scanner devices:

```
# scanimage -L
```

The output should be similar to the following:

```
device `hpaio:/usb/Deskjet_1050_J410_series?serial=XXXXXXXXXXXXXX' is a Hewlett-
```

```
Packard Deskjet_1050_J410_series all-in-one
```

If scanimage(1) is not able to identify the scanner, this message will appear:

```
No scanners were identified. If you were expecting something different,
check that the scanner is plugged in, turned on and detected by the
sane-find-scanner tool (if appropriate). Please read the documentation
which came with this software (README, FAQ, manpages).
```

Once scanimage(1) sees the scanner, the configuration is complete and the scanner is now ready to use.

To activate the service and have it run at boot execute the following command:

```
# sysrc saned_enable=YES
```

While scanimage(1) can be used to perform an image acquisition from the command line, it is often preferable to use a graphical interface to perform image scanning.

Table 22. Graphical scanner programs

Name	License	Package
skanlite	GPL 2.0	graphics/skanlite
GNOME Simple Scan	GPL 3.0	graphics/simple-scan
XSANE	GPL 2.0	graphics/xsane

Chapter 10. Configuring the FreeBSD Kernel

10.1. Synopsis

The kernel is the core of the FreeBSD operating system. It is responsible for managing memory, enforcing security controls, networking, disk access, and much more. While much of FreeBSD is dynamically configurable, some users may wish to configure and compile a custom kernel.

Read this chapter to learn:

- When to build a custom kernel.
- How to take a hardware inventory.
- How to customize a kernel configuration file.
- How to use the kernel configuration file to create and build a new kernel.
- How to install the new kernel.
- How to troubleshoot if things go wrong.

All of the commands listed in the examples in this chapter should be executed as `root`.

10.2. Why Build a Custom Kernel?

Traditionally, FreeBSD used a monolithic kernel. The kernel was one large program, supported a fixed list of devices, and in order to change the kernel's behavior, one had to compile and then reboot into a new kernel.

Today, most of the functionality in the FreeBSD kernel is contained in modules which can be dynamically loaded and unloaded from the kernel as necessary. This allows the running kernel to adapt immediately to new hardware and for new functionality to be brought into the kernel. This is known as a modular kernel.

Occasionally, it is still necessary to perform static kernel configuration. Sometimes the needed functionality is so tied to the kernel that it can not be made dynamically loadable. Some security environments prevent the loading and unloading of kernel modules and require that only needed functionality is statically compiled into the kernel.

Building a custom kernel is often a rite of passage for advanced BSD users. This process, while time consuming, can provide benefits to the FreeBSD system. Unlike the GENERIC kernel, which must support a wide range of hardware, a custom kernel can be stripped down to only provide support for that computer's hardware. This has a number of benefits, such as:

- Faster boot time. Since the kernel will only probe the hardware on the system, the time it takes the system to boot can decrease.
- Lower memory usage. A custom kernel often uses less memory than the GENERIC kernel by omitting unused features and device drivers. This is important because the kernel code remains resident in physical memory at all times, preventing that memory from being used by applications. For this reason, a custom kernel is useful on a system with a small amount of RAM.

- Additional hardware support. A custom kernel can add support for devices which are not present in the GENERIC kernel.



When building a custom kernel, it is important to note that non-default configurations are less thoroughly tested than the GENERIC configuration. While customizing the kernel can provide specific benefits it also increases the risk of encountering build or runtime issues. Custom kernel configurations are recommended only for advanced users who have a specific reason for making changes and are willing to engage in the debugging process if necessary.

Before building a custom kernel, consider the reason for doing so. If there is a need for specific hardware support, it may already exist as a module.

Kernel modules exist in /boot/kernel and may be dynamically loaded into the running kernel using kldload(8). Most kernel drivers have a loadable module and manual page. For example, the ath(4) wireless network driver has the following information in its manual page:

```
Alternatively, to load the driver as a module at boot time, place the
following line in loader.conf(5):

    if_ath_load="YES"
```

Adding `if_ath_load="YES"` to /boot/loader.conf will load this module dynamically at boot time.

In some cases, there is no associated module in /boot/kernel. This is mostly true for certain subsystems.

10.3. Finding the System Hardware

Before editing the kernel configuration file, it is recommended to perform an inventory of the machine's hardware. On a dual-boot system, the inventory can be created from the other operating system. For example, Microsoft®'s Device Manager contains information about installed devices.



Some versions of Microsoft® Windows® have a System icon which can be used to access Device Manager.

If FreeBSD is the only installed operating system, use dmesg(8) to determine the hardware that was found and listed during the boot probe. Most device drivers on FreeBSD have a manual page which lists the hardware supported by that driver. For example, the following lines indicate that the psm(4) driver found a mouse:

```
psm0: <PS/2 Mouse> irq 12 on atkbdc0
psm0: [GIANT-LOCKED]
psm0: [ITHREAD]
psm0: model Generic PS/2 mouse, device ID 0
```

Since this hardware exists, this driver should not be removed from a custom kernel configuration file.

If the output of `dmesg` does not display the results of the boot probe output, instead read the contents of /var/run/dmesg.boot.

Another tool for finding hardware is pciconf(8), which provides more verbose output. For example:

```
% pciconf -lv
ath0@pci0:3:0:0:        class=0x020000 card=0x058a1014 chip=0x1014168c rev=0x01 hdr
=0x00
    vendor     = 'Atheros Communications Inc.'
    device     = 'AR5212 Atheros AR5212 802.11abg wireless'
    class      = network
    subclass   = ethernet
```

This output shows that the ath driver located a wireless Ethernet device.

The `-k` flag of man(1) can be used to provide useful information. For example, it can be used to display a list of manual pages which contain a particular device brand or name:

```
# man -k Atheros
ath(4)                  - Atheros IEEE 802.11 wireless network driver
ath_hal(4)              - Atheros Hardware Access Layer (HAL)
```

Once the hardware inventory list is created, refer to it to ensure that drivers for installed hardware are not removed as the custom kernel configuration is edited.

10.4. The Configuration File

In order to create a custom kernel configuration file and build a custom kernel, the full FreeBSD source tree must first be installed.

If /usr/src/ does not exist or it is empty, source has not been installed. Source can be installed with Git using the instructions in “Using Git”.

Once source is installed, review the contents of /usr/src/sys. This directory contains a number of subdirectories, including those which represent the following supported architectures: amd64, i386, powerpc, and sparc64. Everything inside a particular architecture’s directory deals with that architecture only and the rest of the code is machine independent code common to all platforms. Each supported architecture has a conf subdirectory which contains the GENERIC kernel configuration file for that architecture.

Do not make edits to GENERIC. Instead, copy the file to a different name and make edits to the copy. The convention is to use a name with all capital letters. When maintaining multiple FreeBSD machines with different hardware, it is a good idea to name it after the machine’s hostname. This example creates a copy, named MYKERNEL, of the GENERIC configuration file for the `amd64` architecture:

```
# cd /usr/src/sys/amd64/conf
# cp GENERIC MYKERNEL
```

MYKERNEL can now be customized with any ASCII text editor. The default editor is vi, though an easier editor for beginners, called ee, is also installed with FreeBSD.

The format of the kernel configuration file is simple. Each line contains a keyword that represents a device or subsystem, an argument, and a brief description. Any text after a # is considered a comment and ignored. To remove kernel support for a device or subsystem, put a # at the beginning of the line representing that device or subsystem. Refrain from adding or removing a # for any line you do not fully understand.



It is easy to remove support for a device or option and end up with a broken kernel. For example, if the ata(4) driver is removed from the kernel configuration file, a system using ATA disk drivers may not boot. When in doubt, just leave support in the kernel.

In addition to the brief descriptions provided in this file, additional descriptions are contained in NOTES, which can be found in the same directory as GENERIC for that architecture. For architecture independent options, refer to /usr/src/sys/conf/NOTES.



When finished customizing the kernel configuration file, save a backup copy to a location outside of /usr/src.

Alternately, keep the kernel configuration file elsewhere and create a symbolic link to the file:

```
# cd /usr/src/sys/amd64/conf
# mkdir /root/kernels
# cp GENERIC /root/kernels/MYKERNEL
# ln -s /root/kernels/MYKERNEL
```

An include directive is available for use in configuration files. This allows another configuration file to be included in the current one, making it easy to maintain small changes relative to an existing file. If only a small number of additional options or drivers are required, this allows a delta to be maintained with respect to GENERIC, as seen in this example:

```
include GENERIC
ident MYKERNEL

options         IPFIREWALL
options         DUMMYNET
options         IPFIREWALL_DEFAULT_TO_ACCEPT
options         IPDIVERT
```

Using this method, the local configuration file expresses local differences from a GENERIC kernel.

As upgrades are performed, new features added to GENERIC will also be added to the local kernel unless they are specifically prevented using `nooptions` or `nodevice`. A comprehensive list of configuration directives and their descriptions may be found in config(5).



To build a file which contains all available options, run the following command as `root`:

```
# cd /usr/src/sys/arch/conf && make LINT
```

10.5. Building and Installing a Custom Kernel

Once the edits to the custom configuration file have been saved, the source code for the kernel can be compiled using the following steps:

Procedure: Building a Kernel

1. Change to this directory:

   ```
   # cd /usr/src
   ```

2. Compile the new kernel by specifying the name of the custom kernel configuration file:

   ```
   # make buildkernel KERNCONF=MYKERNEL
   ```

3. Install the new kernel associated with the specified kernel configuration file. This command will copy the new kernel to /boot/kernel/kernel and save the old kernel to /boot/kernel.old/kernel:

   ```
   # make installkernel KERNCONF=MYKERNEL
   ```

4. Shutdown the system and reboot into the new kernel. If something goes wrong, refer to The kernel does not boot.

By default, when a custom kernel is compiled, all kernel modules are rebuilt. To update a kernel faster or to build only custom modules, edit /etc/make.conf before starting to build the kernel.

For example, this variable specifies the list of modules to build instead of using the default of building all modules:

```
MODULES_OVERRIDE = linux acpi
```

Alternately, this variable lists which modules to exclude from the build process:

```
WITHOUT_MODULES = linux acpi sound
```

This variable instructs the build to skip kernel modules and compile only the kernel itself:

```
NO_MODULES = yes
```

The Ports Framework includes drivers and other modules that depend on kernel interfaces, such as graphics/drm-kmod or emulators/virtualbox-ose-kmod. With the PORTS_MODULES variable, every time the kernel is built, the ports containing kernel modules are re-built against the updated sources. This ensures the kernel module stays in-sync with the kernel itself. The kernel and ports trees should be updated together for maximum compatibility. PORTS_MODULES can be added to /etc/make.conf to ensure all kernels you build also rebuild these modules.

```
PORTS_MODULES = drm-kmod virtualbox-ose-kmod
```

Additional variables are available. Refer to make.conf(5) for details.

10.6. If Something Goes Wrong

There are four categories of trouble that can occur when building a custom kernel:

`config` fails

If `config` fails, it will print the line number that is incorrect. As an example, for the following message, make sure that line 17 is typed correctly by comparing it to GENERIC or NOTES:

```
config: line 17: syntax error
```

`make` fails

If `make` fails, it is usually due to an error in the kernel configuration file which is not severe enough for `config` to catch. Review the configuration, and if the problem is not apparent, send an email to the FreeBSD general questions mailing list which contains the kernel configuration file.

The kernel does not boot

If the new kernel does not boot or fails to recognize devices, do not panic! Fortunately, FreeBSD has an excellent mechanism for recovering from incompatible kernels. Simply choose the kernel to boot from at the FreeBSD boot loader. This can be accessed when the system boot menu appears by selecting the "Escape to a loader prompt" option. At the prompt, type `boot kernel.old`, or the name of any other kernel that is known to boot properly.

After booting with a good kernel, check over the configuration file and try to build it again. One helpful resource is /var/log/messages which records the kernel messages from every successful boot. Also, dmesg(8) will print the kernel messages from the current boot.



When troubleshooting a kernel make sure to keep a copy of a kernel that is

known to work, such as GENERIC. This is important because every time a new kernel is installed, kernel.old is overwritten with the last installed kernel, which may or may not be bootable. As soon as possible, move the working kernel by renaming the directory containing the good kernel:

```
# mv /boot/kernel /boot/kernel.bad
# mv /boot/kernel.good /boot/kernel
```

The kernel works, but ps(1) does not

If the kernel version differs from the one that the system utilities have been built with, for example, a kernel built from -CURRENT sources is installed on a -RELEASE system, many system status commands like ps(1) and vmstat(8) will not work. To fix this, recompile and install a world built with the same version of the source tree as the kernel. It is never a good idea to use a different version of the kernel than the rest of the operating system.

Chapter 11. Printing

Putting information on paper is a vital function, despite many attempts to eliminate it. Printing has two basic components. The data must be delivered to the printer, and must be in a form that the printer can understand.

11.1. Quick Start

Basic printing can be set up quickly. The printer must be capable of printing plain ASCII text. For printing to other types of files, see Filters.

1. Create a directory to store files while they are being printed:

```
# mkdir -p /var/spool/lpd/lp
# chown daemon:daemon /var/spool/lpd/lp
# chmod 770 /var/spool/lpd/lp
```

2. As root, create /etc/printcap with these contents:

```
lp:\
lp=/dev/unlpt0:\  ①
sh:\
mx#0:\
sd=/var/spool/lpd/lp:\
lf=/var/log/lpd-errs:
```

① This line is for a printer connected to a USB port.
For a printer connected to a parallel or "printer" port, use:

```
:lp=/dev/lpt0:\
```

For a printer connected directly to a network, use:

```
:lp=:rm=network-printer-name:rp=raw:\
```

Replace *network-printer-name* with the DNS host name of the network printer.

3. Enable LPD by editing /etc/rc.conf, adding this line:

```
lpd_enable="YES"
```

Start the service:

```
# service lpd start
Starting lpd.
```

4. Print a test:

```
# printf "1. This printer can print.\n2. This is the second line.\n" | lpr
```



If both lines do not start at the left border, but "stairstep" instead, see Preventing Stairstepping on Plain Text Printers.

Text files can now be printed with `lpr`. Give the filename on the command line, or pipe output directly into `lpr`.

```
% lpr textfile.txt
% ls -lh | lpr
```

11.2. Printer Connections

Printers are connected to computer systems in a variety of ways. Small desktop printers are usually connected directly to a computer's USB port. Older printers are connected to a parallel or "printer" port. Some printers are directly connected to a network, making it easy for multiple computers to share them. A few printers use a rare serial port connection.

FreeBSD can communicate with all of these types of printers.

USB

USB printers can be connected to any available USB port on the computer.

When FreeBSD detects a USB printer, two device entries are created: /dev/ulpt0 and /dev/unlpt0. Data sent to either device will be relayed to the printer. After each print job, ulpt0 resets the USB port. Resetting the port can cause problems with some printers, so the unlpt0 device is usually used instead. unlpt0 does not reset the USB port at all.

Parallel (IEEE-1284)

The parallel port device is /dev/lpt0. This device appears whether a printer is attached or not, it is not autodetected.

Vendors have largely moved away from these "legacy" ports, and many computers no longer have them. Adapters can be used to connect a parallel printer to a USB port. With such an adapter, the printer can be treated as if it were actually a USB printer. Devices called *print servers* can also be used to connect parallel printers directly to a network.

Serial (RS-232)

Serial ports are another legacy port, rarely used for printers except in certain niche applications.

Cables, connectors, and required wiring vary widely.

For serial ports built into a motherboard, the serial device name is /dev/cuau0 or /dev/cuau1. Serial USB adapters can also be used, and these will appear as /dev/cuaU0.

Several communication parameters must be known to communicate with a serial printer. The most important are *baud rate* or BPS (Bits Per Second) and *parity*. Values vary, but typical serial printers use a baud rate of 9600 and no parity.

Network

Network printers are connected directly to the local computer network.

The DNS hostname of the printer must be known. If the printer is assigned a dynamic address by DHCP, DNS should be dynamically updated so that the host name always has the correct IP address. Network printers are often given static IP addresses to avoid this problem.

Most network printers understand print jobs sent with the LPD protocol. A print queue name can also be specified. Some printers process data differently depending on which queue is used. For example, a `raw` queue prints the data unchanged, while the `text` queue adds carriage returns to plain text.

Many network printers can also print data sent directly to port 9100.

11.2.1. Summary

Wired network connections are usually the easiest to set up and give the fastest printing. For direct connection to the computer, USB is preferred for speed and simplicity. Parallel connections work but have limitations on cable length and speed. Serial connections are more difficult to configure. Cable wiring differs between models, and communication parameters like baud rate and parity bits must add to the complexity. Fortunately, serial printers are rare.

11.3. Common Page Description Languages

Data sent to a printer must be in a language that the printer can understand. These languages are called Page Description Languages, or PDLs.

ASCII

Plain ASCII text is the simplest way to send data to a printer. Characters correspond one to one with what will be printed: an A in the data prints an A on the page. Very little formatting is available. There is no way to select a font or proportional spacing. The forced simplicity of plain ASCII means that text can be printed straight from the computer with little or no encoding or translation. The printed output corresponds directly with what was sent.

Some inexpensive printers cannot print plain ASCII text. This makes them more difficult to set up, but it is usually still possible.

PostScript®

PostScript® is almost the opposite of ASCII. Rather than simple text, a PostScript® program is a set of instructions that draw the final document. Different fonts and graphics can be used.

However, this power comes at a price. The program that draws the page must be written. Usually this program is generated by application software, so the process is invisible to the user.

Inexpensive printers sometimes leave out PostScript® compatibility as a cost-saving measure.

PCL (Printer Command Language)

PCL is an extension of ASCII, adding escape sequences for formatting, font selection, and printing graphics. Many printers provide PCL5 support. Some support the newer PCL6 or PCLXL. These later versions are supersets of PCL5 and can provide faster printing.

Host-Based

Manufacturers can reduce the cost of a printer by giving it a simple processor and very little memory. These printers are not capable of printing plain text. Instead, bitmaps of text and graphics are drawn by a driver on the host computer and then sent to the printer. These are called *host-based* printers.

Communication between the driver and a host-based printer is often through proprietary or undocumented protocols, making them functional only on the most common operating systems.

11.3.1. Converting PostScript® to Other PDLs

Many applications from the Ports Collection and FreeBSD utilities produce PostScript® output. This table shows the utilities available to convert that into other common PDLs:

Table 23. Output PDLs

Output PDL	Generated By	Notes
PCL or PCL5	print/ghostscript9-base	-sDEVICE=ljet4 for monochrome, -sDEVICE=cljet5 for color
PCLXL or PCL6	print/ghostscript9-base	-sDEVICE=pxlmono for monochrome, -sDEVICE=pxlcolor for color
ESC/P2	print/ghostscript9-base	-sDEVICE=uniprint
XQX	print/foo2zjs	

11.3.2. Summary

For the easiest printing, choose a printer that supports PostScript®. Printers that support PCL are the next preferred. With print/ghostscript9-base, these printers can be used as if they understood PostScript® natively. Printers that support PostScript® or PCL directly almost always support direct printing of plain ASCII text files also.

Line-based printers like typical inkjets usually do not support PostScript® or PCL. They often can print plain ASCII text files. print/ghostscript9-base supports the PDLs used by some of these printers. However, printing an entire graphic-based page on these printers is often very slow due to the large amount of data to be transferred and printed.

Host-based printers are often more difficult to set up. Some cannot be used at all because of proprietary PDLs. Avoid these printers when possible. The particular PDL used by various models of printers can be found at http://www.openprinting.org/printers.

11.4. Direct Printing

For occasional printing, files can be sent directly to a printer device without any setup. For example, a file called sample.txt can be sent to a USB printer:

```
# cp sample.txt /dev/unlpt0
```

Direct printing to network printers depends on the abilities of the printer, but most accept print jobs on port 9100, and nc(1) can be used with them. To print the same file to a printer with the DNS hostname of *netlaser*:

```
# nc netlaser 9100 < sample.txt
```

11.5. LPD (Line Printer Daemon)

Printing a file in the background is called *spooling*. A spooler allows the user to continue with other programs on the computer without waiting for the printer to slowly complete the print job.

FreeBSD includes a spooler called lpd(8). Print jobs are submitted with lpr(1).

11.5.1. Initial Setup

A directory for storing print jobs is created, ownership is set, and the permissions are set to prevent other users from viewing the contents of those files:

```
# mkdir -p /var/spool/lpd/lp
# chown daemon:daemon /var/spool/lpd/lp
# chmod 770 /var/spool/lpd/lp
```

Printers are defined in /etc/printcap. An entry for each printer includes details like a name, the port where it is attached, and various other settings. Create /etc/printcap with these contents:

```
lp:\                ①
    :lp=/dev/unlpt0:\   ②
    :sh:\           ③
    :mx#0:\         ④
    :sd=/var/spool/lpd/lp:\ ⑤
    :lf=/var/log/lpd-errs:  ⑥
```

① The name of this printer. lpr(1) sends print jobs to the lp printer unless another printer is specified with -P, so the default printer should be named lp.

② The device where the printer is connected. Replace this line with the appropriate one for the connection type shown here.

③ Suppress the printing of a header page at the start of a print job.

④ Do not limit the maximum size of a print job.

⑤ The path to the spooling directory for this printer. Each printer uses its own spooling directory.

⑥ The log file where errors on this printer will be reported.

After creating /etc/printcap, use chkprintcap(8) to test it for errors:

```
# chkprintcap
```

Fix any reported problems before continuing.

Enable lpd(8) in /etc/rc.conf:

```
lpd_enable="YES"
```

Start the service:

```
# service lpd start
```

11.5.2. Printing with lpr(1)

Documents are sent to the printer with `lpr`. A file to be printed can be named on the command line or piped into `lpr`. These two commands are equivalent, sending the contents of doc.txt to the default printer:

```
% lpr doc.txt
% cat doc.txt | lpr
```

Printers can be selected with `-P`. To print to a printer called *laser*:

```
% lpr -Plaser doc.txt
```

11.5.3. Filters

The examples shown so far have sent the contents of a text file directly to the printer. As long as the printer understands the content of those files, output will be printed correctly.

Some printers are not capable of printing plain text, and the input file might not even be plain text.

Filters allow files to be translated or processed. The typical use is to translate one type of input, like plain text, into a form that the printer can understand, like PostScript® or `PCL`. Filters can also be

used to provide additional features, like adding page numbers or highlighting source code to make it easier to read.

The filters discussed here are *input filters* or *text filters*. These filters convert the incoming file into different forms. Use su(1) to become `root` before creating the files.

Filters are specified in /etc/printcap with the `if=` identifier. To use /usr/local/libexec/lf2crlf as a filter, modify /etc/printcap like this:

```
lp:\
    :lp=/dev/unlpt0:\
    :sh:\
    :mx#0:\
    :sd=/var/spool/lpd/lp:\
    :if=/usr/local/libexec/lf2crlf:\   ①
    :lf=/var/log/lpd-errs:
```

① `if=` identifies the *input filter* that will be used on incoming text.



The backslash *line continuation* characters at the end of the lines in printcap entries reveal that an entry for a printer is really just one long line with entries delimited by colon characters. An earlier example can be rewritten as a single less-readable line:

```
lp:lp=/dev/unlpt0:sh:mx#0:sd=/var/spool/lpd/lp:if=/usr/local/libexec/lf
2crlf:lf=/var/log/lpd-errs:
```

11.5.3.1. Preventing Stairstepping on Plain Text Printers

Typical FreeBSD text files contain only a single line feed character at the end of each line. These lines will "stairstep" on a standard printer:

```
A printed file looks
                   like the steps of a staircase
                                              scattered by the wind
```

A filter can convert the newline characters into carriage returns and newlines. The carriage returns make the printer return to the left after each line. Create /usr/local/libexec/lf2crlf with these contents:

```
#!/bin/sh
CR=$'\r'
/usr/bin/sed -e "s/$/${CR}/g"
```

Set the permissions and make it executable:

```
# chmod 555 /usr/local/libexec/lf2crlf
```

Modify /etc/printcap to use the new filter:

```
:if=/usr/local/libexec/lf2crlf:\
```

Test the filter by printing the same plain text file. The carriage returns will cause each line to start at the left side of the page.

11.5.3.2. Fancy Plain Text on PostScript® Printers with print/enscript

GNUEnscript converts plain text files into nicely-formatted PostScript® for printing on PostScript® printers. It adds page numbers, wraps long lines, and provides numerous other features to make printed text files easier to read. Depending on the local paper size, install either print/enscript-letter or print/enscript-a4 from the Ports Collection.

Create /usr/local/libexec/enscript with these contents:

```
#!/bin/sh
/usr/local/bin/enscript -o -
```

Set the permissions and make it executable:

```
# chmod 555 /usr/local/libexec/enscript
```

Modify /etc/printcap to use the new filter:

```
:if=/usr/local/libexec/enscript:\
```

Test the filter by printing a plain text file.

11.5.3.3. Printing PostScript® to PCL Printers

Many programs produce PostScript® documents. However, inexpensive printers often only understand plain text or PCL. This filter converts PostScript® files to PCL before sending them to the printer.

Install the Ghostscript PostScript® interpreter, print/ghostscript9-base, from the Ports Collection.

Create /usr/local/libexec/ps2pcl with these contents:

```
#!/bin/sh
/usr/local/bin/gs -dSAFER -dNOPAUSE -dBATCH -q -sDEVICE=ljet4 -sOutputFile=- -
```

Set the permissions and make it executable:

```
# chmod 555 /usr/local/libexec/ps2pcl
```

PostScript® input sent to this script will be rendered and converted to PCL before being sent on to the printer.

Modify /etc/printcap to use this new input filter:

```
:if=/usr/local/libexec/ps2pcl:\
```

Test the filter by sending a small PostScript® program to it:

```
% printf "%%\!PS \n /Helvetica findfont 18 scalefont setfont \
72 432 moveto (PostScript printing successful.) show showpage \004" | lpr
```

11.5.3.4. Smart Filters

A filter that detects the type of input and automatically converts it to the correct format for the printer can be very convenient. The first two characters of a PostScript® file are usually %!. A filter can detect those two characters. PostScript® files can be sent on to a PostScript® printer unchanged. Text files can be converted to PostScript® with Enscript as shown earlier. Create /usr/local/libexec/psif with these contents:

```
#!/bin/sh
#
#  psif - Print PostScript or plain text on a PostScript printer
#
IFS="" read -r first_line
first_two_chars=`expr "$first_line" : '\(..\)'`

case "$first_two_chars" in
%!)
    # %! : PostScript job, print it.
    echo "$first_line" && cat && exit 0
    exit 2
    ;;
*)
    # otherwise, format with enscript
    ( echo "$first_line"; cat ) | /usr/local/bin/enscript -o - && exit 0
    exit 2
    ;;
esac
```

Set the permissions and make it executable:

```
# chmod 555 /usr/local/libexec/psif
```

Modify /etc/printcap to use this new input filter:

```
:if=/usr/local/libexec/psif:\
```

Test the filter by printing PostScript® and plain text files.

11.5.4. Multiple Queues

The entries in /etc/printcap are really definitions of *queues*. There can be more than one queue for a single printer. When combined with filters, multiple queues provide users more control over how their jobs are printed.

As an example, consider a networked PostScript® laser printer in an office. Most users want to print plain text, but a few advanced users want to be able to print PostScript® files directly. Two entries can be created for the same printer in /etc/printcap:

```
textprinter:\
    :lp=9100@officelaser:\
    :sh:\
    :mx#0:\
    :sd=/var/spool/lpd/textprinter:\
    :if=/usr/local/libexec/enscript:\
    :lf=/var/log/lpd-errs:

psprinter:\
    :lp=9100@officelaser:\
    :sh:\
    :mx#0:\
    :sd=/var/spool/lpd/psprinter:\
    :lf=/var/log/lpd-errs:
```

Documents sent to `textprinter` will be formatted by the /usr/local/libexec/enscript filter shown in an earlier example. Advanced users can print PostScript® files on `psprinter`, where no filtering is done.

This multiple queue technique can be used to provide direct access to all kinds of printer features. A printer with a duplexer could use two queues, one for ordinary single-sided printing, and one with a filter that sends the command sequence to enable double-sided printing and then sends the incoming file.

11.5.5. Monitoring and Controlling Printing

Several utilities are available to monitor print jobs and check and control printer operation.

11.5.5.1. lpq(1)

lpq(1) shows the status of a user's print jobs. Print jobs from other users are not shown.

Show the current user's pending jobs on a single printer:

```
% lpq -Plp
Rank   Owner      Job  Files                                 Total Size
1st    jsmith     0    (standard input)                      12792 bytes
```

Show the current user's pending jobs on all printers:

```
% lpq -a
lp:
Rank   Owner      Job  Files                                 Total Size
1st    jsmith     1    (standard input)                      27320 bytes

laser:
Rank   Owner      Job  Files                                 Total Size
1st    jsmith     287  (standard input)                      22443 bytes
```

11.5.5.2. lprm(1)

lprm(1) is used to remove print jobs. Normal users are only allowed to remove their own jobs. `root` can remove any or all jobs.

Remove all pending jobs from a printer:

```
# lprm -Plp -
dfA002smithy dequeued
cfA002smithy dequeued
dfA003smithy dequeued
cfA003smithy dequeued
dfA004smithy dequeued
cfA004smithy dequeued
```

Remove a single job from a printer. lpq(1) is used to find the job number.

```
% lpq
Rank   Owner      Job  Files                                 Total Size
1st    jsmith     5    (standard input)                      12188 bytes

% lprm -Plp 5
dfA005smithy dequeued
cfA005smithy dequeued
```

11.5.5.3. lpc(8)

lpc(8) is used to check and modify printer status. `lpc` is followed by a command and an optional printer name. `all` can be used instead of a specific printer name, and the command will be applied to all printers. Normal users can view status with lpc(8). Only `root` can use commands which modify printer status.

Show the status of all printers:

```
% lpc status all
lp:
    queuing is enabled
    printing is enabled
    1 entry in spool area
    printer idle
laser:
    queuing is enabled
    printing is enabled
    1 entry in spool area
    waiting for laser to come up
```

Prevent a printer from accepting new jobs, then begin accepting new jobs again:

```
# lpc disable lp
lp:
    queuing disabled
# lpc enable lp
lp:
    queuing enabled
```

Stop printing, but continue to accept new jobs. Then begin printing again:

```
# lpc stop lp
lp:
    printing disabled
# lpc start lp
lp:
    printing enabled
    daemon started
```

Restart a printer after some error condition:

```
# lpc restart lp
lp:
    no daemon to abort
    printing enabled
```

```
    daemon restarted
```

Turn the print queue off and disable printing, with a message to explain the problem to users:

```
# lpc down lp Repair parts will arrive on Monday
lp:
    printer and queuing disabled
    status message is now: Repair parts will arrive on Monday
```

Re-enable a printer that is down:

```
# lpc up lp
lp:
    printing enabled
    daemon started
```

See lpc(8) for more commands and options.

11.5.6. Shared Printers

Printers are often shared by multiple users in businesses and schools. Additional features are provided to make sharing printers more convenient.

11.5.6.1. Aliases

The printer name is set in the first line of the entry in /etc/printcap. Additional names, or *aliases*, can be added after that name. Aliases are separated from the name and each other by vertical bars:

```
lp|repairsprinter|salesprinter:\
```

Aliases can be used in place of the printer name. For example, users in the Sales department print to their printer with

```
% lpr -Psalesprinter sales-report.txt
```

Users in the Repairs department print to *their* printer with

```
% lpr -Prepairsprinter repairs-report.txt
```

All of the documents print on that single printer. When the Sales department grows enough to need their own printer, the alias can be removed from the shared printer entry and used as the name of a new printer. Users in both departments continue to use the same commands, but the Sales documents are sent to the new printer.

11.5.6.2. Header Pages

It can be difficult for users to locate their documents in the stack of pages produced by a busy shared printer. *Header pages* were created to solve this problem. A header page with the user name and document name is printed before each print job. These pages are also sometimes called *banner* or *separator* pages.

Enabling header pages differs depending on whether the printer is connected directly to the computer with a USB, parallel, or serial cable, or is connected remotely over a network.

Header pages on directly-connected printers are enabled by removing the `:sh:\` (Suppress Header) line from the entry in /etc/printcap. These header pages only use line feed characters for new lines. Some printers will need the /usr/share/examples/printing/hpif filter to prevent stairstepped text. The filter configures PCL printers to print both carriage returns and line feeds when a line feed is received.

Header pages for network printers must be configured on the printer itself. Header page entries in /etc/printcap are ignored. Settings are usually available from the printer front panel or a configuration web page accessible with a web browser.

11.5.7. References

Example files: /usr/share/examples/printing/.

The *4.3BSD Line Printer Spooler Manual*, /usr/share/doc/smm/07.lpd/paper.ascii.gz.

Manual pages: printcap(5), lpd(8), lpr(1), lpc(8), lprm(1), lpq(1).

11.6. Other Printing Systems

Several other printing systems are available in addition to the built-in lpd(8). These systems offer support for other protocols or additional features.

11.6.1. CUPS (Common UNIX® Printing System)

CUPS is a popular printing system available on many operating systems. Using CUPS on FreeBSD is documented in a separate article: CUPS

11.6.2. HPLIP

Hewlett Packard provides a printing system that supports many of their inkjet and laser printers. The port is print/hplip. The main web page is at https://developers.hp.com/hp-linux-imaging-and-printing. The port handles all the installation details on FreeBSD. Configuration information is shown at https://developers.hp.com/hp-linux-imaging-and-printing/install.

11.6.3. LPRng

LPRng was developed as an enhanced alternative to lpd(8). The port is sysutils/LPRng. For details and documentation, see https://lprng.sourceforge.net/.

Chapter 12. Linux Binary Compatibility

12.1. Synopsis

FreeBSD provides **optional** binary compatibility with Linux®, commonly referred to as Linuxulator, allowing users to install and run unmodified Linux binaries. It is available for the x86 (both 32 and 64 bit) and AArch64 architectures. Some Linux-specific operating system features are not yet supported; this mostly happens with functionality specific to hardware or related to system management, such as cgroups or namespaces.

Before reading this chapter:

- Know how to install additional third-party software.

Read this chapter to learn:

- How to enable Linux binary compatibility on a FreeBSD system.
- How to install additional Linux shared libraries.
- How to install Linux applications on a FreeBSD system.
- The implementation details of Linux compatibility in FreeBSD.

12.2. Configuring Linux Binary Compatibility

By default, linux(4) binary compatibility is not enabled.

To enable the Linux ABI at boot time, execute the following command:

```
# sysrc linux_enable="YES"
```

Once enabled, it can be started without rebooting by executing the following command:

```
# service linux start
```

This is enough for statically linked Linux binaries to work.

The Linux service will load necessary kernel modules and mount filesystems expected by Linux applications under /compat/linux. They can be started in the same way native FreeBSD binaries can; they behave almost exactly like native processes and can be traced and debugged the usual way.

The current content of /compat/linux can be checked executing the following command:

```
# ls -l /compat/linux/
```

The output should be similar to the following:

```
total 1
dr-xr-xr-x  13 root  wheel  512 Apr 11 19:12 dev
dr-xr-xr-x   1 root  wheel    0 Apr 11 21:03 proc
dr-xr-xr-x   1 root  wheel    0 Apr 11 21:03 sys
```

12.3. Linux userlands

Linux software requires more than just an ABI to work. In order to run Linux software a Linux userland must be installed first.



If all that is wanted is to run some software already included in the Ports tree, it can be installed via package manager and pkg(8) will automatically setup the required Linux userland.

For example, to install Sublime Text 4, along with all the Linux libraries it depends on, run this command:

```
# pkg install linux-sublime-text4
```

12.3.1. Rocky Linux Base System from FreeBSD Packages

To install the Rocky Linux 9 userland execute the following command:

```
# pkg install linux_base-rl9
```

emulators/linux_base-rl9 will place the base system derived from Rocky Linux 9 into /compat/linux.

After installing the package, the contents of /compat/linux can be verified by running the following command to check that the Rocky Linux userland has been installed:

```
# ls -l /compat/linux/
```

The output should be similar to the following:

```
total 36
drwxr-xr-x   2 root wheel  512 Oct  9 17:28 afs
lrwxr-xr-x   1 root wheel    7 May 16  2022 bin -> usr/bin
drwxr-xr-x   3 root wheel  512 Oct  9 17:28 dev
drwxr-xr-x  24 root wheel 1536 Oct  9 17:28 etc
lrwxr-xr-x   1 root wheel    7 May 16  2022 lib -> usr/lib
lrwxr-xr-x   1 root wheel    9 May 16  2022 lib64 -> usr/lib64
drwxr-xr-x   2 root wheel  512 Oct  9 17:28 opt
```

```
drwxr-xr-x   2 root wheel  512 Oct  9 17:28 proc
lrwxr-xr-x   1 root wheel    8 Oct  1 03:11 run -> /var/run
lrwxr-xr-x   1 root wheel    8 May 16  2022 sbin -> usr/sbin
drwxr-xr-x   2 root wheel  512 Oct  9 17:28 srv
drwxr-xr-x   2 root wheel  512 Oct  9 17:28 sys
drwxr-xr-x   8 root wheel  512 Oct  9 17:28 usr
drwxr-xr-x  16 root wheel  512 Oct  9 17:28 var
```

12.3.2. CentOS Base System from FreeBSD Packages



emulators/linux_base-c7 has been deprecated following the deprecation of the upstream project. This means emulators/linux_base-c7 will not receive security updates. The use of the Rocky Linux Base System is recommended unless 32-bit compatibility is needed.

To install the CentOS userland execute the following command:

```
# pkg install linux_base-c7
```

emulators/linux_base-c7 will place the base system derived from CentOS 7 into /compat/linux.

After installing the package, the contents of /compat/linux can be verified by running the following command to check that the CentOS userland has been installed:

```
# ls -l /compat/linux/
```

The output should be similar to the following:

```
total 30
lrwxr-xr-x   1 root  wheel    7 Apr 11  2018 bin -> usr/bin
drwxr-xr-x  13 root  wheel  512 Apr 11 21:10 dev
drwxr-xr-x  25 root  wheel   64 Apr 11 21:10 etc
lrwxr-xr-x   1 root  wheel    7 Apr 11  2018 lib -> usr/lib
lrwxr-xr-x   1 root  wheel    9 Apr 11  2018 lib64 -> usr/lib64
drwxr-xr-x   2 root  wheel    2 Apr 11 21:10 opt
dr-xr-xr-x   1 root  wheel    0 Apr 11 21:25 proc
lrwxr-xr-x   1 root  wheel    8 Feb 18 02:10 run -> /var/run
lrwxr-xr-x   1 root  wheel    8 Apr 11  2018 sbin -> usr/sbin
drwxr-xr-x   2 root  wheel    2 Apr 11 21:10 srv
dr-xr-xr-x   1 root  wheel    0 Apr 11 21:25 sys
drwxr-xr-x   8 root  wheel    9 Apr 11 21:10 usr
drwxr-xr-x  16 root  wheel   17 Apr 11 21:10 var
```

12.3.3. Debian / Ubuntu Base System with debootstrap

An alternative way of providing Linux shared libraries is by using sysutils/debootstrap. This has the advantage of providing a full Debian or Ubuntu distribution.

To install debootstrap execute the following command:

```
# pkg install debootstrap
```

debootstrap(8) needs linux(4) ABI enabled. Once enabled, execute the following command to install Ubuntu or Debian in /compat/ubuntu:

```
# debootstrap jammy /compat/ubuntu
```



While it is technically possible to install into /compat/linux instead, it's discouraged due to possible clashes with CentOS-based packages. Instead, derive the directory name from the distribution or version name, e.g., /compat/ubuntu.

The output should be similar to the following:

```
I: Retrieving InRelease
I: Checking Release signature
I: Valid Release signature (key id F6ECB3762474EDA9D21B7022871920D1991BC93C)
I: Retrieving Packages
I: Validating Packages
I: Resolving dependencies of required packages...
I: Resolving dependencies of base packages...
I: Checking component main on http://archive.ubuntu.com/ubuntu...
[...]
I: Configuring console-setup...
I: Configuring kbd...
I: Configuring ubuntu-minimal...
I: Configuring libc-bin...
I: Configuring ca-certificates...
I: Base system installed successfully.
```

Then set up mounts in /etc/fstab.



If the contents of the home directory should be shared and to be able to run X11 applications, /home and /tmp should be mounted in the linux compat area using nullfs(5) for loopback.

The following example can be added to /etc/fstab:

```
# Device        Mountpoint              FStype          Options
Dump    Pass#
```

```
devfs           /compat/ubuntu/dev      devfs           rw,late
0       0
tmpfs           /compat/ubuntu/dev/shm  tmpfs
rw,late,size=1g,mode=1777    0       0
fdescfs         /compat/ubuntu/dev/fd   fdescfs
rw,late,linrdlnk             0       0
linprocfs       /compat/ubuntu/proc     linprocfs       rw,late
0       0
linsysfs        /compat/ubuntu/sys      linsysfs        rw,late
0       0
/tmp            /compat/ubuntu/tmp      nullfs          rw,late
0       0
/home           /compat/ubuntu/home     nullfs          rw,late
0       0
```

Then execute mount(8):

```
# mount -al
```

To access the system using chroot(8) execute the following command:

```
# chroot /compat/ubuntu /bin/bash
```

Then uname(1) can be executed to check the Linux environment:

```
# uname -s -r -m
```

The output should be similar to the following:

```
Linux 5.15.0 x86_64
```

Once inside the chroot, the system behaves as in a normal Ubuntu installation. While systemd doesn't work, the service(8) command works as usual.



To add the package repositories missing from defaults edit the file /compat/ubuntu/etc/apt/sources.list.

For amd64 the following example can be used:

```
deb http://archive.ubuntu.com/ubuntu jammy main universe restricted
multiverse
deb http://security.ubuntu.com/ubuntu/ jammy-security universe
multiverse restricted main
deb http://archive.ubuntu.com/ubuntu jammy-backports universe
multiverse restricted main
```

```
deb http://archive.ubuntu.com/ubuntu jammy-updates universe multiverse
restricted main
```

For arm64 this other example can be used:

```
deb http://ports.ubuntu.com/ubuntu-ports bionic main universe
restricted multiverse
```

12.4. Advanced Topics

A list of all Linux-related sysctl(8) knobs can be found in linux(4).

Some applications require specific filesystems to be mounted.

This is normally handled by the /etc/rc.d/linux script but can be disabled at boot executing the following command:

```
sysrc linux_mounts_enable="NO"
```

Filesystems mounted by the rc script will not work for Linux processes inside chroots or jails; if needed, configure them in /etc/fstab:

```
devfs      /compat/linux/dev      devfs      rw,late                    0  0
tmpfs      /compat/linux/dev/shm  tmpfs      rw,late,size=1g,mode=1777  0  0
fdescfs    /compat/linux/dev/fd   fdescfs    rw,late,linrdlnk           0  0
linprocfs  /compat/linux/proc     linprocfs  rw,late                    0  0
linsysfs   /compat/linux/sys      linsysfs   rw,late                    0  0
```

Since the Linux binary compatibility layer has gained support for running both 32- and 64-bit Linux binaries, it is no longer possible to link the emulation functionality statically into a custom kernel.

12.4.1. Installing Additional Libraries Manually



For base system subdirectories created with debootstrap(8), use the instructions above instead.

If a Linux application complains about missing shared libraries after configuring Linux binary compatibility, determine which shared libraries the Linux binary needs and install them manually.

From a Linux system using the same CPU architecture, `ldd` can be used to determine which shared libraries the application needs.

For example, to check which shared libraries `linuxdoom` needs, run this command from a Linux system that has Doom installed:

```
% ldd linuxdoom
```

The output should be similar to the following:

```
libXt.so.3 (DLL Jump 3.1) => /usr/X11/lib/libXt.so.3.1.0
libX11.so.3 (DLL Jump 3.1) => /usr/X11/lib/libX11.so.3.1.0
libc.so.4 (DLL Jump 4.5pl26) => /lib/libc.so.4.6.29
```

Then, copy all the files in the last column of the output from the Linux system into /compat/linux on the FreeBSD system. Once copied, create symbolic links to the names in the first column.

This example will result in the following files on the FreeBSD system:

```
/compat/linux/usr/X11/lib/libXt.so.3.1.0
/compat/linux/usr/X11/lib/libXt.so.3 -> libXt.so.3.1.0
/compat/linux/usr/X11/lib/libX11.so.3.1.0
/compat/linux/usr/X11/lib/libX11.so.3 -> libX11.so.3.1.0
/compat/linux/lib/libc.so.4.6.29
/compat/linux/lib/libc.so.4 -> libc.so.4.6.29
```

If a Linux shared library already exists with a matching major revision number to the first column of the `ldd` output, it does not need to be copied to the file named in the last column, as the existing library should work. It is advisable to copy the shared library if it is a newer version, though. The old one can be removed, as long as the symbolic link points to the new one.

For example, these libraries already exist on the FreeBSD system:

```
/compat/linux/lib/libc.so.4.6.27
/compat/linux/lib/libc.so.4 -> libc.so.4.6.27
```

and `ldd` indicates that a binary requires a later version:

```
libc.so.4 (DLL Jump 4.5pl26) -> libc.so.4.6.29
```

Since the existing library is only one or two versions out of date in the last digit, the program should still work with the slightly older version. However, it is safe to replace the existing libc.so with the newer version:

```
/compat/linux/lib/libc.so.4.6.29
/compat/linux/lib/libc.so.4 -> libc.so.4.6.29
```

Generally, one will need to look for the shared libraries that Linux binaries depend on only the first few times that a Linux program is installed on FreeBSD. After a while, there will be a sufficient set

of Linux shared libraries on the system to be able to run newly installed Linux binaries without any extra work.

12.4.2. Branding Linux ELF Binaries

The FreeBSD kernel uses several methods to determine if the binary to be executed is a Linux one: it checks the brand in the ELF file header, looks for known ELF interpreter paths and checks ELF notes; finally, by default, unbranded ELF executables are assumed to be Linux anyway.

Should all those methods fail, an attempt to execute the binary might result in error message:

```
% ./my-linux-elf-binary
```

The output should be similar to the following:

```
ELF binary type not known
Abort
```

To help the FreeBSD kernel distinguish between a FreeBSD ELF binary and a Linux binary, use brandelf(1):

```
% brandelf -t Linux my-linux-elf-binary
```

12.4.3. Installing a Linux RPM Based Application

To install a Linux RPM-based application, first install the archivers/rpm4 package or port. Once installed, `root` can use this command to install a .rpm:

```
# cd /compat/linux
# rpm2cpio < /path/to/linux.archive.rpm | cpio -id
```

If necessary, `brandelf` the installed ELF binaries. Note that this will prevent a clean uninstall.

12.4.4. Configuring the Hostname Resolver

If DNS does not work or this error appears:

```
resolv+: "bind" is an invalid keyword resolv+:
"hosts" is an invalid keyword
```

configure /compat/linux/etc/host.conf as follows:

```
order hosts, bind
```

```
multi on
```

This specifies that /etc/hosts is searched first and DNS is searched second. When /compat/linux/etc/host.conf does not exist, Linux applications use /etc/host.conf in the host system but they complain since that file does not exist in FreeBSD. Remove `bind` if a name server is not configured using /etc/resolv.conf.

12.4.5. Miscellaneous

More information on how binary compatibility works with Linux® can be found in the article Linux emulation in FreeBSD.

Chapter 13. WINE

13.1. Synopsis

WINE, which stands for Wine Is Not an Emulator, is technically a software translation layer. It allows installing and running software written for Windows® on FreeBSD (and other) systems.

It operates by intercepting system calls, or requests from the software to the operating system, and translating them from Windows® calls to calls that FreeBSD understands. It will also translate any responses as needed into what the Windows® software is expecting. So in some ways, it *emulates* a Windows® environment, in that it provides many of the resources Windows® applications are expecting.

However, it is not an emulator in the traditional sense. Many of these solutions operate by constructing an entirely separate computer using software processes in place of hardware. Virtualization (such as that provided by the emulators/qemu port) operates in this way. One of the benefits of this approach is the ability to install a full version of the OS in question to the emulator. It means that the environment will not look any different to applications than a real machine, and chances are good that everything will work on it. The downside to this approach is the fact that software acting as hardware is inherently slower than actual hardware. The computer built in software (called the *guest*) requires resources from the real machine (the *host*), and holds on to those resources for as long as it is running.

The WINE Project, on the other hand, is much lighter on system's resources. It will translate system calls on the fly, so while it is difficult to be as fast as a real Windows® computer, it can come very close. On the other hand, WINE is trying to keep up with a moving target in terms of all the different system calls and other functionality it needs to support. As a result there may be applications that do not work as expected on WINE, will not work at all, or will not even install to begin with.

At the end of the day, WINE provides another option to try to get a particular Windows® software program running on FreeBSD. It can always serve as the first option which, if successful, offers a good experience without unnecessarily depleting the host FreeBSD system's resources.

This chapter will describe:

- How to install WINE on a FreeBSD system.
- How WINE operates, and how it is different from other alternatives like virtualization.
- How to fine-tune WINE to the specific needs of some applications.
- How to install GUI helpers for WINE.
- Common tips and solutions for using WINE on FreeBSD.
- Considerations for WINE on FreeBSD in terms of the multi-user environment.

Before reading this chapter, it will be useful to:

- Understand the basics of UNIX® and FreeBSD.

- Know how to install FreeBSD.
- Know how to set up a network connection.
- Know how to install additional third-party software.

13.2. WINE Overview & Concepts

WINE is a complex system, so before running it on a FreeBSD system it is worth gaining an understanding of what it is and how it works.

13.2.1. What is WINE?

As mentioned in the Synopsis for this chapter, WINE is a compatibility layer that allows Windows® applications to run on other operating systems. In theory, it means these programs should run on systems like FreeBSD, macOS, and Android.

When WINE runs a Windows® executable, two things occur:

- Firstly, WINE implements an environment that mimics that of various versions of Windows®. For example, if an application requests access to a resource such as RAM, WINE has a memory interface that looks and acts (as far as the application is concerned) like Windows®.
- Then, once that application makes use of that interface, WINE takes the incoming request for space in memory and translates it to something compatible with the host system. In the same way when the application retrieves that data, WINE facilitates fetching it from the host system and passing it back to the Windows® application.

13.2.2. WINE and the FreeBSD System

Installing WINE on a FreeBSD system will entail a few different components:

- FreeBSD applications for tasks such as running the Windows® executables, configuring the WINE sub-system, or compiling programs with WINE support.
- A large number of libraries that implement the core functions of Windows® (for example /lib/wine/api-ms-core-memory-l1-1-1.dll.so, which is part of the aforementioned memory interface).
- A number of Windows® executables, which are (or mimic) common utilities (such as /lib/wine/notepad.exe.so, which provides the standard Windows® text editor).
- Additional Windows® assets, in particular fonts (like the Tahoma font, which is stored in share/wine/fonts/tahoma.ttf in the install root).

13.2.3. Graphical Versus Text Mode/Terminal Programs in WINE

As an operating system where terminal utilities are "first-class citizens," it is natural to assume that WINE will contain extensive support for text-mode program. However, the majority of applications for Windows®, especially the most popular ones, are designed with a graphical user interface (GUI) in mind. Therefore, WINE's utilities are designed by default to launch graphical programs.

However, there are three methods available to run these so-called Console User Interface (CUI) programs:

- The *Bare Streams* approach will display the output directly to standard output.
- The *wineconsole* utility can be used with either the *user* or *curses* backend to utilize some of the enhancements the WINE system provides for CUI applications.

These approaches are described in greater detail on the WINE Wiki.

13.2.4. WINE Derivative Projects

WINE itself is a mature open source project, so it is little surprise it is used as the foundation of more complex solutions.

13.2.4.1. Commercial WINE Implementations

A number of companies have taken WINE and made it a core of their own, proprietary products (WINE's LGPL license permits this). Two of the most famous of these are as follows:

- Codeweavers CrossOver

This solution provides a simplified "one-click" installation of WINE, which contains additional enhancements and optimizations (although the company contributes many of these back upstream to the WINE project). One area of focus for Codeweavers is to make the most popular applications install and run smoothly.

While the company once produced a native FreeBSD version of their CrossOver solution, it appears to have long been abandoned. While some resources (such as a dedicated forum) are still present, they also have seen no activity for some time.

- Steam Proton

Gaming company Steam also uses WINE to enable Windows® games to install and run on other systems. it is primary target is Linux-based systems, though some support exists for macOS as well.

While Steam does not offer a native FreeBSD client, there are several options for using the Linux® client using FreeBSD's Linux Compatibility Layer.

13.2.4.2. WINE Companion Programs

In addition to proprietary offerings, other projects have released applications designed to work in tandem with the standard, open source version of WINE. The goals for these can range from making installation easier to offering easy ways to get popular software installed.

These solutions are covered in greater detail in the later section on GUI frontends, and include the following:

- winetricks
- Mizutamari

13.2.5. Alternatives to WINE

For FreeBSD users, some alternatives to using WINE are as follows:

- Dual-Booting: A straightforward option is to run desired Windows® applications natively on that OS. This of course means exiting FreeBSD in order to boot Windows®, so this method is not feasible if access to programs in both systems is required simultaneously.
- Virtual Machines: Virtual Machines (VMs), as mentioned earlier in this chapter, are software processes that emulate full sets of hardware, on which additional operating systems (including Windows®) can be installed and run. Modern tools make VMs easy to create and manage, but this method comes at a cost. A good portion of the host systems resources must be allocated to each VM, and those resources cannot be reclaimed by the host as long as the VM is running. A few examples of VM managers include the open source solutions qemu, bhyve, and VirtualBox. See the chapter on Virtualization for more detail.
- Remote Access: Like many other UNIX®-like systems, FreeBSD can run a variety of applications enabling users to remotely access Windows® computers and use their programs or data. In addition to clients such as xrdp that connect to the standard Windows® Remote Desktop Protocol, other open source standards such as vnc can also be used (provided a compatible server is present on the other side).

13.3. Installing WINE on FreeBSD

WINE can be installed via the pkg tool, or by compiling the port(s).

13.3.1. WINE Prerequistes

Before installing WINE itself, it is useful to have the following pre-requisites installed.

- A GUI

Most Windows® programs are expecting to have a graphical user interface available. If WINE is installed without one present, its dependencies will include the Wayland compositor, and so a GUI will be installed along with WINE. But it is useful to have the GUI of choice installed, configured, and working correctly before installing WINE.

- wine-gecko

The Windows® operating system has for some time had a default web browser pre-installed: Internet Explorer. As a result, some applications work under the assumption that there will always be something capable of displaying web pages. In order to provide this functionality, the WINE layer includes a web browser component using the Mozilla project's Gecko engine. When WINE is first launched it will offer to download and install this, and there are reasons users might want it do so (these will be covered in a later chapter). But they can also install it prior to installing WINE, or alongside the install of WINE proper.

Install this package with the following:

```
# pkg install wine-gecko
```

Alternately, compile the port with the following:

```
# cd /usr/ports/emulator/wine-gecko
# make install
```

- wine-mono

This port installs the MONO framework, an open source implementation of Microsoft's .NET. Including this with the WINE installation will make it that much more likely that any applications written in .NET will install and run on the system.

To install the package:

```
# pkg install wine-mono
```

To compile from the ports collection:

```
# cd /usr/ports/emulator/wine-mono
# make install
```

13.3.2. Installing WINE via FreeBSD Package Repositories

With the pre-requisites in place, install WINE via package with the following command:

```
# pkg install wine
```

Alternately compile the WINE sub-system from source with the following:

```
# cd /usr/ports/emulator/wine
# make install
```

13.3.3. Concerns of 32- Versus 64-Bit in WINE Installations

Like most software, Windows® applications made the upgrade from the older 32-bit architecture to 64 bits. And most recent software is written for 64-bit operating systems, although modern OSes can sometimes continue to run older 32-bit programs as well. FreeBSD is no different, having had support for 64-bit since the 5.x series.

However, using old software no longer supported by default is a common use for emulators, and users commonly turn to WINE to play games and use other programs that do not run properly on modern hardware. Fortunately, FreeBSD can support all three scenarios:

- On modern, 64-bit machine and want to run 64-bit Windows® software, simply install the ports mentioned in the above sections. The ports system will automatically install the 64-bit version.
- Alternately, users might have an older 32-bit machine that they do not want to run with its original, now non-supported software. They can install the 32-bit (i386) version of FreeBSD, then install the ports in the above sections.

13.4. Running a First WINE Program on FreeBSD

Now that WINE is installed, the next step is to try it out by running a simple program. An easy way to do this is to download a self-contained application, i.e., one can simply unpack and run without any complex installation process.

So-called "portable" versions of applications are good choices for this test, as are programs that run with only a single executable file.

13.4.1. Running a Program from the Command Line

There are two different methods to launch a Windows program from the terminal. The first, and most straightforward is to navigate to the directory containing the program's executable (.EXE) and issue the following:

```
% wine program.exe
```

For applications that take command-line arguments, add them after the executable as usual:

```
% wine program2.exe -file file.txt
```

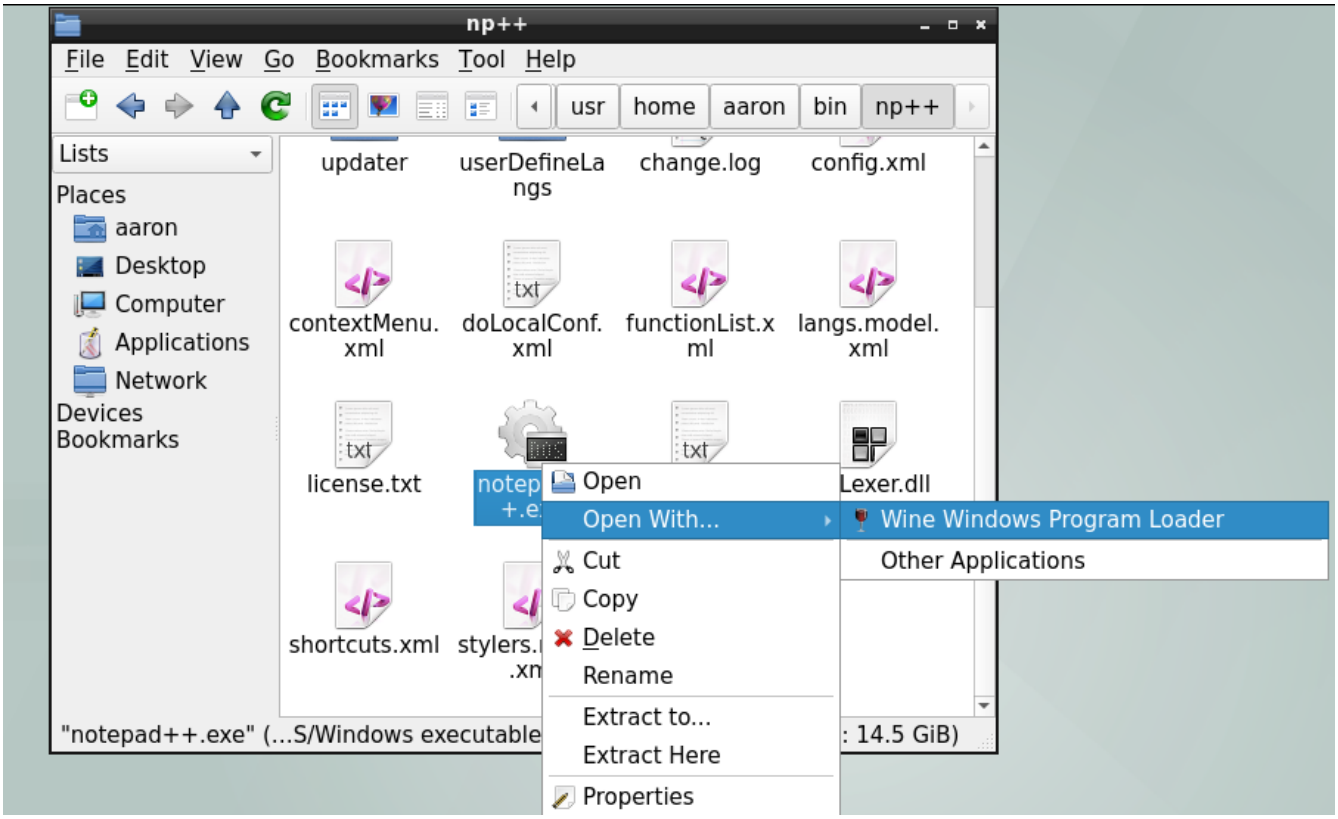
Alternately, supply the full path to the executable to use it in a script, for example:

```
% wine /home/user/bin/program.exe
```

13.4.2. Running a Program from a GUI

After installation graphical shells should be updated with new associations for Windows executable (.EXE) files. It will now be possible to browse the system using a file manager, and launch the Windows application in the same way as other files and programs (either a single- or double-click, depending on the desktop's settings).

On most desktops, check to make sure this association is correct by right-clicking on the file, and looking for an entry in the context menu to open the file. One of the options (hopefully the default one) will be with the **Wine Windows Program Loader**, as shown in the below screenshot:



In the event the program does not run as expected, try launching it from the command line and review any messages displayed in the terminal to troubleshoot.

In the event WINE is not the default application for .EXE files after install, check the MIME associate for this extension in the current desktop environment, graphical shell, or file manager.

13.5. Configuring WINE Installation

With an understanding of what WINE is and how it works at a high level, the next step to effectively using it on FreeBSD is becoming familiar with its configuration. The following sections will describe the key concept of the *WINE prefix*, and illustrate how it is used to control the behavior of applications run through WINE.

13.5.1. WINE Prefixes

A WINE *prefix* is a directory, usually located beneath the default location of $HOME/.wine though it can be located elsewhere. The prefix is a set of configurations and support files used by the wine to configure and run the Windows® environment a given application needs. By default, a brand new WINE installation will create the following structure when first launched by a user:

- .update-timestamp: contains the last modified date of file /usr/share/wine/wine.inf. It is used by WINE to determine if a prefix is out of date, and automatically update it if needed.
- dosdevices/: contains information on mappings of Windows® resources to resources on the host FreeBSD system. For example, after a new WINE installation, this should contain at least two entries which enable access to the FreeBSD filesystem using Windows®-style drive letters:
  - c:@: A link to drive_c described below.
  - z:@: A link to the root directory of the system.

- drive_c/: emulates the main (i.e., C:) drive of a Windows® system. It contains a directory structure and associated files mirroring that of standard Windows® systems. A fresh WINE prefix will contain Windows® 10 directories such as *Users* and *Windows* that holds the OS itself. Furthermore, applications installed within a prefix will be located in either *Program Files* or *Program Files (x86)*, depending on their architecture.
- system.reg: This Registry file contains information on the Windows® installation, which in the case of WINE is the environment in drive_c.
- user.reg: This Registry file contains the current user's personal configurations, made either by various software or through the use of the Registry Editor.
- userdef.reg: This Registry file is a default set of configurations for newly-created users.

13.5.2. Creating and Using WINE Prefixes

While WINE will create a default prefix in the user's $HOME/.wine/, it is possible to set up multiple prefixes. There are a few reasons to do this:

- The most common reason is to emulate different versions of Windows®, according to the compatibility needs of the software in question.
- In addition, it is common to encounter software that does not work correctly in the default environment, and requires special configuration. it is useful to isolate these in their own, custom prefixes, so the changes do not impact other applications.
- Similarly, copying the default or "main" prefix into a separate "testing" one in order to evaluate an application's compatibility can reduce the chance of corruption.

Creating a prefix from the terminal requires the following command:

```
% WINEPREFIX="/home/username/.wine-new" winecfg
```

This will run the `winecfg` program, which can be used to configure wine prefixes (more on this in a later section). But by providing a directory path value for the `WINEPREFIX` environment variable, a new prefix is created at that location if one does not already exist.

Supplying the same variable to the wine program will similarly cause the selected program to be run with the specified prefix:

```
% WINEPREFIX="/home/username/.wine-new" wine program.exe
```

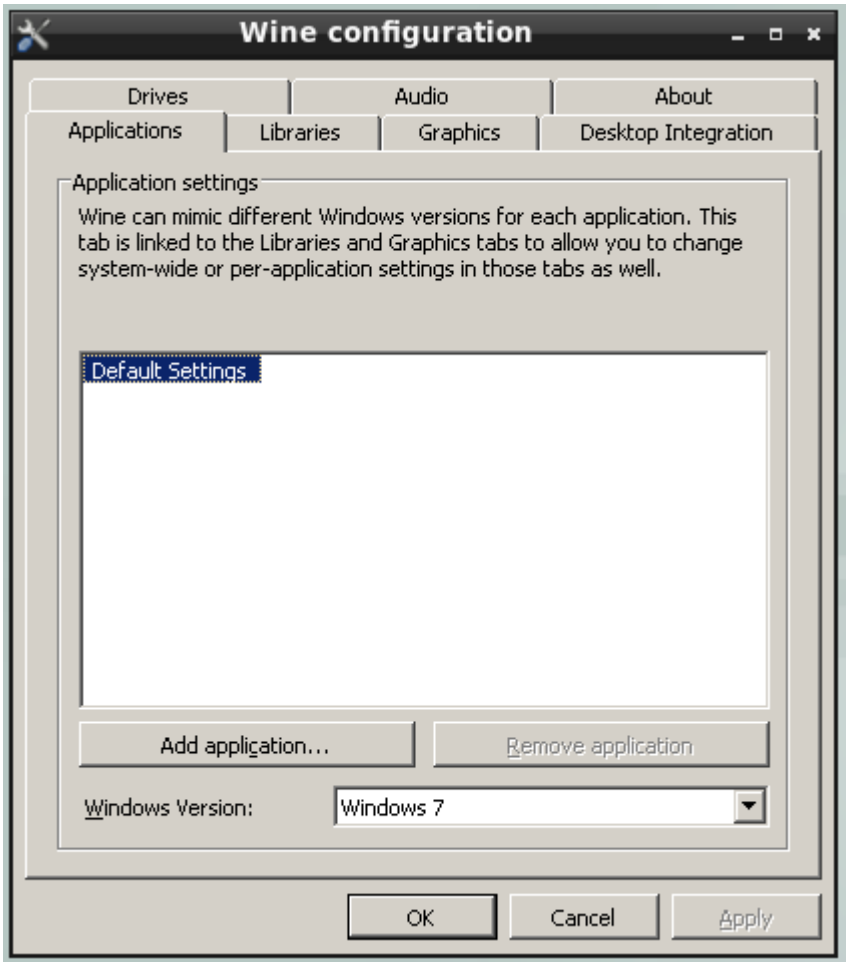
13.5.3. Configuring WINE Prefixes with winecfg

As described above WINE includes a tool called `winecfg` to configure prefixes from within a GUI. It contains a variety of functions, which are detailed in the sections below. When `winecfg` is run from within a prefix, or provided the location of a prefix within the `WINEPREFIX` variable, it enables the configuration of the selected prefix as described in the below sections.

Selections made on the *Applications* tab will affect the scope of changes made in the *Libraries* and

Graphics tabs, which will be limited to the application selected. See the section on Using Winecfg in the WINE Wiki for more details.

13.5.3.1. Applications

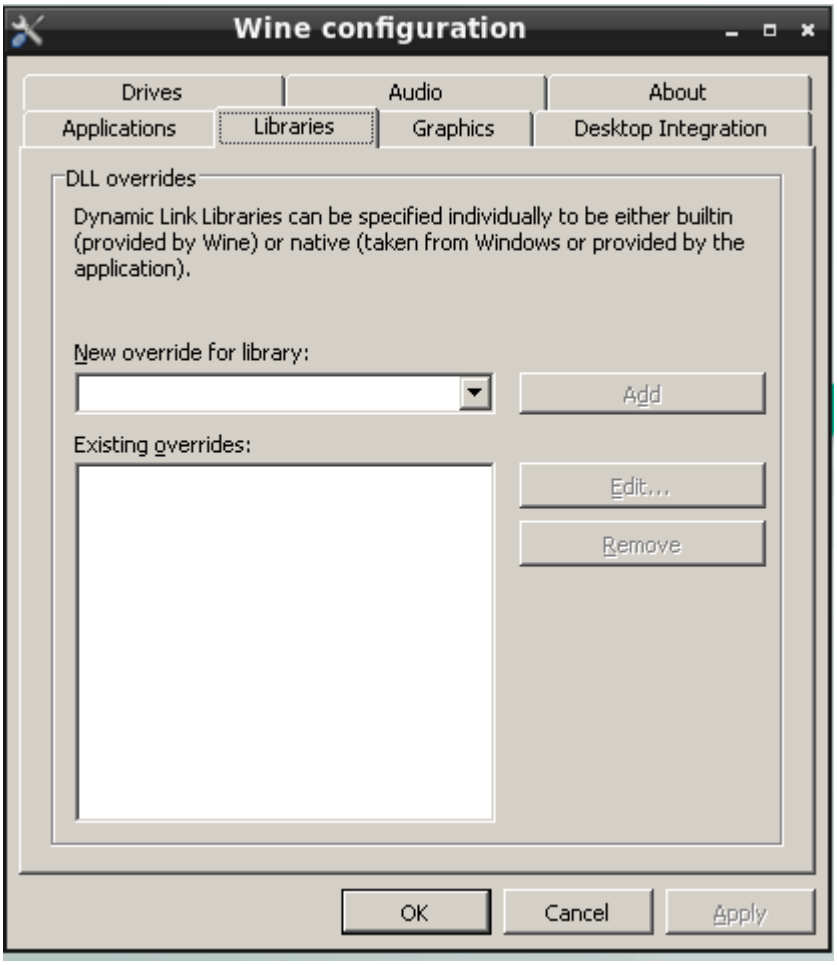


The *Applications* contains controls enabling the association of programs with a particular version of Windows®. On first start-up the *Application settings* section will contain a single entry: *Default Settings*. This corresponds to all the default configurations of the prefix, which (as the disabled *Remove application* button implies) cannot be deleted.

But additional applications can be added with the following process:

1. Click the *Add application* button.
2. Use the provided dialog to select the desired program's executable.
3. Select the version of Windows® to be used with the selected program.

13.5.3.2. Libraries



WINE provides a set of open source library files as part of its distribution that provide the same functions as their Windows® counterparts. However, as noted earlier in this chapter, the WINE project is always trying to keep pace with new updates to these libraries. As a result, the versions that ship with WINE may be missing functionality that the latest Windows® programs are expecting.

However, `winecfg` makes it possible specify overrides for the built-in libraries, particularly there is a version of Windows® available on the same machine as the host FreeBSD installation. For each library to be overridden, do the following:

1. Open the *New override for library* drop-down and select the library to be replaced.
2. Click the *Add* button.
3. The new override will appear in the *Existing overrides* list, notice the *native, builtin* designation in parentheses.
4. Click to select the library.
5. Click the *Edit* button.
6. Use the provided dialog to select a corresponding library to be used in place of the built-in one.

Be sure to select a file that is truly the corresponding version of the built-in one, otherwise there may be unexpected behavior.